

Aide à la réussite 06/11

5 november 2023

Les exercices présentés dans ce document viennent des examens et tests présents sur la page Moodle du cours *Programmation et Algorithmique I*.

1 Récursivité

Déplacements dans une maison (30 août 2010)

Considérez une maison comportant un ensemble de pièces. Certaines de ces pièces sont reliées entre elles par une porte, et l'on peut donc passer de l'une à l'autre en empruntant cette porte. On suppose que les différentes pièces de la maison sont stockées dans une liste de chaînes de caractères, par exemple ['hall', 'salon', 'cuisine']. On suppose également que les différentes portes sont stockées dans une liste de couples (' p_1 ', ' p_2 ') indiquant que l'on peut passer de la pièce ' p_1 ' à la pièce ' p_2 '. Notez bien que les portes ne s'ouvrent pas forcément dans les deux sens. Ainsi, une porte (' p_1 ', ' p_2 ') n'implique pas forcément que l'on peut passer de la pièce ' p_2 ' à la pièce ' p_1 '.

On vous demande d'implémenter un algorithme qui, étant donné une pièce de départ, une pièce d'arrivée ainsi qu'une liste des portes de la maison, retourne le chemin (pièce par pièce) permettant de se rendre de la pièce de départ à la pièce d'arrivée. Par exemple, si la maison contient les pièces ['a', 'b', 'c', 'd'] ainsi que les portes [('a', 'b'), ('b', 'c'), ('a', 'c'), ('b', 'a'), ('d', 'b')]) et que l'on vous demande de passer de la pièce 'd' à la pièce 'c', l'algorithme peut retourner la liste suivante : ['d', 'b', 'c'] (notez que l'on peut également passer de 'd' à 'c' en suivant le chemin ['d', 'b', 'a', 'c'])

Question

Implémentez les fonctions récursives suivantes dans l'ordre :

1. Une fonction récursive **peut_aller(piece1, piece2, portes, eviter)** qui retourne la valeur booléenne vrai si et seulement s'il est possible, en partant de la **piece1**, de rejoindre la **piece2** en respectant les **portes** (une liste de tuples), sans passer deux fois par la même pièce. Le paramètre **eviter** consiste en une liste des pièces à éviter. Lors du premier appel récursif, cette liste est vide.

2. Une fonction récursive **chemin(piece1, piece2, portes, eviter)** qui retourne un chemin sous la forme d'une liste des pièces à traverser (celles de départ et d'arrivée incluses) pour aller de la **piece1** à la **piece2**. Si aucun chemin n'existe, la fonction retourne None. Les paramètres **portes** et **eviter** sont identiques à ceux de la fonction précédente.

Aire d'un polygone (13 juin 2012)

Soit P un polygone convexe à n côtés ($n > 2$) dans le plan $\mathbb{R}^2$. L'objectif de cet exercice est de concevoir un algorithme récursif qui calcule l'aire de P.

Considérons $p_1, \dots, p_n$ les sommets de P énumérés dans le sens des aiguilles d'une montre (le choix du premier sommet p_1 n'a pas d'importance). Par exemple, la liste $[(0,0), (0,1), (1,1), (1,0)]$ représente un carré de côté 1.

Le cas de base de l'algorithme récursif est réalisé lorsque le polygone P possède 3 sommets.

Dans ce cas, le polygone est un triangle et son aire est calculée comme suit :

- notons $p_1 = (x_1, y_1), p_2 = (x_2, y_2)$ et $p_3 = (x_3, y_3)$;
- l'aire du triangle est égale à

$$\frac{1}{2} \left| \det \begin{pmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{pmatrix} \right| = \frac{1}{2} |(x_2 - x_1) \cdot (y_3 - y_1) - (x_3 - x_1) \cdot (y_2 - y_1)|$$

Pour le cas récursif, nous avons que le nombre de sommets du polygone P est supérieur ou égal à 4. L'idée d'une étape récursive est de calculer l'aire de P à partir de deux polygones convexes à n_1 et n_2 côtés, avec $n_1 < n$ et $n_2 < n$, construits comme suit (illustration en figure 1) :

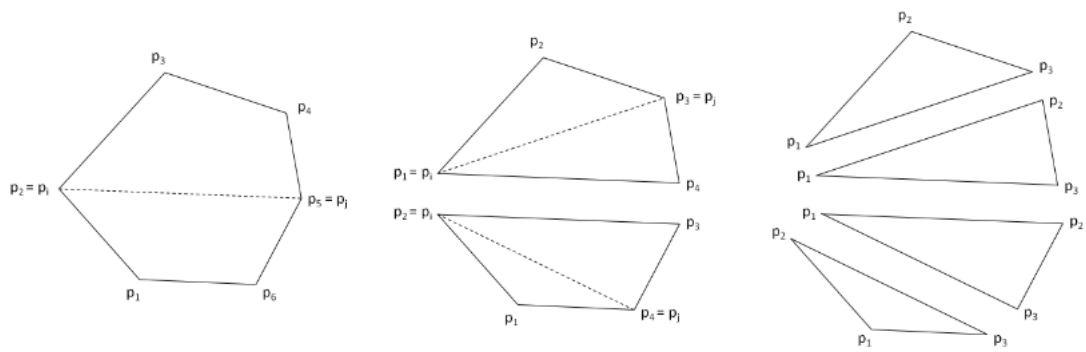
- on choisit deux sommets distincts p_i et p_j ($1 \leq i < j \leq n$) de P **non consécutifs** d'un point de vue géométrique (donc p_1 et p_n sont considérés consécutifs) ;
- n considère les deux polygones suivants : l'un formé par les sommets $p_i, \dots, p_j$ et l'autre formé par les sommets $p_j, \dots, p_n, p_1, \dots, p_i$
- (récurrence) l'aire de P est égale à la somme des aires des deux polygones considérés au point précédent.

Question

Donnez le code python d'une fonction récursive *aire(liste)* qui étant donné une liste *liste* de tuples(couples), calcule l'aire du polygone formé par les couples contenus dans *liste*, en utilisant la méthode décrite précédemment.

Préconditions :

- *liste* contient des couples (tuples à deux composantes);



Figur 1: Exemple d'étapes de récurrence pour un polygone à 6 sommets.

- *liste* représente les sommets d'un polygone convexe et ceux-ci sont énumérés dans le sens des aiguilles d'une montre.

Affichage récursif (20 novembre 2009)

Pour cette question, il vous est demandé d'implémenter des fonctions récursives. Vous **ne pouvez pas** utiliser de boucle. Vous **ne pouvez pas** utiliser l'opérateur `*` sur les chaînes de caractères. Pour la résolution d'une fonction, vous pouvez utiliser les autres fonctions déjà écrites.

Question

Donnez le code Python des fonctions suivantes :

1. Une fonction récursive **multiply(s, nb)** qui prend en entrée **s** une chaîne de caractères et **nb** un nombre entier positif (précondition). Son rôle est de retourner une chaîne de caractères qui est la concaténation de **nb** fois la chaîne **s**.
Par exemple, `multiply('a', 5)` retourne la chaîne `aaaaa`.

2. Une fonction récursive **print_from_middle(s, nb = 1)** qui prend en entrée **s** une chaîne de caractères de taille impaire et **nb** un nombre entier positif impair (1 par défaut). Ce sont des préconditions. Son rôle est d'afficher à l'écran sur une ligne les **nb** caractères au milieu de la chaîne **s**, sur une autre ligne, les **(nb+2)** caractères au milieu de la chaîne **s**, sur la ligne suivante les **(nb+4)** caractères au milieu de la chaîne **s**, et ainsi de suite, jusqu'à afficher la chaîne **s** entière. Par exemple, `print_from_middle('recursivite')` affiche ceci à l'écran :

```
s
rsi
ursiv
cursivi
```

ecursivit
 recursive

3. Une fonction récursive **print_pyramid(s, nb = 1)** qui prend en entrée **s** une chaîne de caractères de taille impaire et **nb** un nombre entier positif impair (1 par défaut). Ce sont des préconditions. Son rôle est d'afficher une sorte de pyramide avec les caractères de la chaîne. Sur la première ligne, elle affiche les **nb** caractères au milieu de la chaîne **s** avec, de part et d'autre, un certain nombre de fois le caractère '-'. Sur la deuxième ligne, elle affiche les **(nb+2)** caractères au milieu de la chaîne **s** avec, de part et d'autre, un certain nombre de fois le caractère '-'. Sur la ligne suivante, elle affiche les **(nb+4)** caractères au milieu de la chaîne **s** avec, de part et d'autre, un certain nombre de fois le caractère '-', et ainsi de suite, jusqu'à afficher la chaîne **s** entière (sans aucun caractère '-'). Le nombre de caractères '-' à afficher par ligne doit être choisi afin de permettre de garder un affichage centré (voir exemple). Par exemple, **print_pyramid('recursive')** affiche ceci à l'écran :

```
—s—
—rsi—
—ursiv—
—cursivi—
—ecursivit—
recursive
```

2 Listes et chaînes de caractères

Opérations matricielles (3 novembre 2010)

Nous représentons une matrice ($n \times m$) par une liste (de longueur n) de listes (de longueur m). Nous obtenons alors la représentation suivante :

$$\begin{pmatrix} a_{0,0} & \dots & a_{0,m-1} \\ \dots & \dots & \dots \\ \dots & \dots & \dots \\ a_{n-1,0} & \dots & a_{n-1,m-1} \end{pmatrix} \quad [[a_{0,0}, \dots, a_{0,m-1}], [a_{1,0}, \dots, a_{1,m-1}], \dots, [a_{n-1,0}, \dots, a_{n-1,m-1}]]$$

Pour accéder à l'élément $a_{i,j}$ ($0 \leq i < n$ et $0 \leq j < m$) d'une matrice **A**, nous écrivons donc : **A[i][j]**

Questions

Donnez le code Python des fonctions suivantes :

1. Une fonction **add(m1, m2)** qui retourne une nouvelle matrice correspondant à la somme des matrices m1 et m2. Si les tailles des matrices ne sont pas compatibles, retourne None.
2. Une fonction **multiply(m1, m2)** qui retourne une nouvelle matrice correspondant au produit des matrices m1 et m2. Si les tailles des matrices ne sont pas compatibles, retourne None.

Compression de chaînes (10 juin 2011)

La compression est une technique visant à transformer sans perte d'informations un ensemble de données en un nouvel ensemble de données en vue de diminuer la taille nécessaire pour stocker ces informations.

Il vous est demandé d'implémenter une technique de compression de chaînes de caractères de la manière suivante : si au moins deux lettres successives sont présentes dans la chaîne de caractères, elles sont remplacées par le nombre de fois que la lettre apparaît successivement suivi de la lettre elle-même. Par exemple, la chaîne 'aaabaccccaa' est compressée en '3aba4c2a' et la chaîne 'ab11a3b' est décompressée en 'abaaaaaaaaaabb'.

Préconditions : les chaînes de caractères qui peuvent être compressées ne contiennent que des **lettres minuscules** (et donc ni chiffres, ni espaces, ni caractères spéciaux). Il s'ensuit que les chaînes de caractères qui peuvent être décompressées ne contiennent que des lettres minuscules et des chiffres.

Questions

- Etant donnée une chaîne compressée de longueur n, quelle sera la longueur minimale et maximale de la version décompressée de cette chaîne de caractères ? Expliquez et donnez un exemple ;
- implémentez la fonction **compresser(chaine)** qui retourne une chaîne de caractères correspondant à la compression de la chaîne de caractères passée en paramètre ;
- implémentez la fonction **decompresser(chaine)** qui retourne une chaîne de caractères correspondant à la décompression de la chaîne de caractères passée en paramètre.

3 Boucle

Ordonnanceur de tâches (20 novembre 2009)

Un ordonnanceur de tâches a pour objectif de trouver un ordre dans lequel différentes tâches numérotées peuvent être effectuées tout en respectant certaines contraintes de précedence.

Une contrainte de précédence (i, j) stipule que la tâche numéro j doit être réalisée avant d'effectuer la tâche numéro i .

A titre d'exemple, considérez un système informatique comprenant 4 tâches devant être réalisées, numérotées de 0 à 3. Les contraintes de précédence de ce système sont les suivantes : $(2, 0), (2, 1), (0, 1), (3, 2)$. Un ordonnanceur de tâches trouvera l'ordre de réalisation des tâches suivant : 1, 0, 2, 3.

Un système de tâches à ordonnancer peut être encodé sous la forme d'une matrice carrée de taille $n \times n$ où n est le nombre de tâches à ordonnancer. Dans cette matrice, l'élément en position (i, j) a la valeur *True* si une contrainte de précédence (i, j) a été spécifiée, *False* sinon. Ainsi, dans notre exemple, le système serait encodé sous la forme de la matrice 4×4 suivante :

$$\begin{pmatrix} False & True & False & False \\ False & False & False & False \\ True & True & False & False \\ False & False & True & False \end{pmatrix}$$

Il n'est pas toujours possible de trouver un ordre de réalisation des tâches. Par exemple, dans un système à deux tâches, si les contraintes de précédence sont $(0, 1), (1, 0)$, il n'existe aucun ordre permettant de réaliser les tâches tout en respectant les contraintes.

Questions

Donnez le code Python des fonctions suivantes :

1. Une fonction **creer_matrice(n_taches, contraintes)** qui, étant donnés un nombre **n_taches** de tâches et une liste **contraintes** de couples représentant la liste des contraintes d'un système, retourne la matrice du système correspondante. Cette fonction a pour précondition que **n_taches** ≥ 1 .
2. Une fonction **ordonnanceur(matrice)** qui, étant donnée une matrice encodant le système à ordonnancer, retourne une liste d'entiers représentant un ordre dans lequel les tâches peuvent être réalisées. Dans notre exemple, cette fonction retournerait $[1, 0, 2, 3]$. Cette fonction a pour précondition que la matrice fournie en paramètre encode un système pour lequel il existe toujours un ordre de réalisation des tâches. Notez que la fonction **ordonnanceur** ne doit pas avoir d'effet de bord.

Recherche dichotomique (13 juin 2012)

La recherche dichotomique est un moyen très efficace de rechercher un élément donné au sein d'une séquence triée d'éléments. Il s'agit d'un processus itératif visant à découper à

chaque étape l'espace de recherche en deux parties égales (si possible).

À chaque étape de l'algorithme, la position centrale de l'espace de recherche courant est calculée et la valeur de l'élément à cette position est comparée avec l'élément recherché.

1. L'élément en position centrale est l'élément recherché, auquel cas l'algorithme s'arrête et retourne la position de cet élément ;
2. L'élément en position centrale est plus grand que l'élément recherché, auquel cas l'espace de recherche courant est réduit à sa partie à gauche de la position centrale ;
3. L'élément en position centrale est plus petit que l'élément recherché, auquel cas l'espace de recherche courant est réduit à sa partie à droite de la position centrale.

Ces étapes se répètent successivement jusqu'à ce que l'élément soit trouvé (cas 1.) ou que l'espace de recherche soit réduit à une taille suffisamment petite (par exemple, un seul ou deux éléments).

À titre d'exemple, considérez que l'on recherche l'élément 4 au sein de la séquence triée suivante :

[0, 1, 2, 3, 5, 6, 8, 10, 11, 14, 15].

Lors de la première étape de cet algorithme, l'espace de recherche considéré couvre les éléments 0 à 15. L'élément 6, en position centrale, étant plus grand que l'élément 4 recherché, l'espace de recherche est réduit aux éléments 0 à 5. Lors de la seconde étape de l'algorithme, l'élément central de l'espace de recherche réduit de 0 à 5 est comparé avec l'élément recherché. Puisque 4 est supérieur à 2, l'espace de recherche est réduit à la portion de droite, à savoir les éléments 3 à 5. Cet espace de recherche étant suffisamment petit, l'algorithme peut déduire que l'élément recherché 4 n'était pas présent dans la séquence initiale.

Questions

Donnez le code Python d'une fonction **recherche_dicho(seq, x)** implémentant une méthode de recherche dichotomique itérative (non-réursive) et qui retourne une position (un indice) à laquelle l'élément x se trouve dans la liste triée par ordre croissant **seq**. La fonction retourne la valeur -1 si x n'est pas dans **seq**. Le fait que **seq** soit triée est une précondition. Cette fonction ne peut pas être réursive, elle ne peut pas avoir d'effets de bord.

4 Invariant de boucle

Multiplication d'entiers (13 juin 2012)

Voici le code Python d'une fonction dont le but est de calculer le produit de deux nombres entiers a et b (la numérotation des lignes ne fait pas partie du code mais peut être utilisée pour faire référence à une instruction dans votre réponse).

```
1. def mult(a, b):
2.     r = 0
3.     s = b
4.     while s != 0:
5.         r = r + a
6.         s = s - 1
7.     return r
```

Questions

1. On suppose que a et b sont des entiers. Quelles sont les conditions qu'il faut obligatoirement imposer sur a et/ou b pour que la fonction `mult` s'arrête ? Prouvez que sous ces conditions, la fonction s'arrête toujours.
2. Parmi les assertions suivantes, quelles sont celles (s'il y en a) qui sont des invariants de la boucle `while` ? (vous ne devez pas justifier votre réponse à ce stade, juste donner la liste des invariants corrects) :
 - (S1) $r = a \times b$
 - (S2) $s \leq b$
 - (S3) $r = a \times (b - s)$
 - (S4) $r = a \times s$
3. Prouvez que, sous les conditions énoncées au point 1, la fonction **mult** calcule effectivement le produit de a et de b (vous devez donc ici choisir un invariant de boucle ; prouver rigoureusement qu'il est effectivement un invariant de boucle ; puis en déduire des arguments prouvant l'exactitude de la fonction).

Factorisation en nombres premiers (10 novembre 2015)

Pour rappel, un nombre entier $n \geq 2$ est premier si et seulement s'il n'admet comme diviseurs que 1 et lui-même. La décomposition en facteurs premiers d'un nombre x est un ensemble de nombres premiers tel que le produit de ces nombres est égal à x . Étant donné un nombre entier $x \geq 2$, cette décomposition est unique. Considérez le code de la fonction **decomposition(x)** suivante qui permet de calculer la décomposition en facteurs premiers d'un nombre entier

$x \geq 2$. Son principe est de diviser x par i tant que c'est possible, c'est à dire tant que x est divisible par i .

```
1. def decomposition(x):
2.     i = 2
3.     r = []
4.     while x > 1:
5.         while x % i == 0:
6.             x = x / i
7.             r.append(i)
8.             i = i + 1
9.     return r
```

Voici un exemple de résultats obtenus par la fonction :

```
>>> print decomposition(84)
[2, 2, 3, 7]
>>> decomposition(243)
[3, 3, 3, 3, 3]
>>> print decomposition(17)
[17]
```

Questions

On suppose (précondition) que le nombre x donné en entrée est un entier $x \geq 2$. Pour chacune des assertions suivantes, indiquez s'il s'agit d'un invariant de la boucle externe ($\text{while } x \geq 1$). Si oui, prouvez qu'il est respecté, sinon, expliquez pourquoi ce n'est pas un invariant de boucle.

1. r contient tous les nombres premiers plus petits que i .
2. r contient tous les diviseurs de x plus petits que i .
3. r contient tous les diviseurs premiers de x qui sont strictement inférieurs à i .